



**TECNOLÓGICO
NACIONAL DE MÉXICO®**



Instituto Tecnológico de Tepic

Ingeniería En Sistemas Computacionales

Docente: Irving Yair Nava Hernandez

Materia: Programación Web

Tema 4 Programación del lado del servidor

Nombre:

Erick Alejandro Miramontes Huerta

No.Ctrl: 22400625

Fecha de entrega: 13 de julio del 2026

Tepic, Nay.

¿Qué es JavaScript?

JavaScript (JS) es un lenguaje de programación de alto nivel, interpretado y multiparadigma que se utiliza principalmente para agregar interactividad a las páginas web. Fue creado en 1995 por Brendan Eich y actualmente es uno de los tres pilares del desarrollo web junto con HTML y CSS. Además de ejecutarse en navegadores, también puede utilizarse en servidores mediante Node.js



1. Características básicas del lenguaje

JavaScript posee varias características que lo convierten en uno de los lenguajes más utilizados en el desarrollo web:

- Interpretado: El código se ejecuta directamente por el navegador o el entorno de ejecución sin necesidad de compilar previamente.
- Dinámico: El tipo de dato de una variable puede cambiar durante la ejecución del programa.
- Multiparadigma: Permite programar utilizando distintos estilos como programación orientada a objetos, funcional e imperativa.
- Orientado a objetos: Trabaja con objetos y clases para representar información.
- Basado en prototipos: Los objetos pueden heredar propiedades y métodos de otros objetos.
- Multiplataforma: Funciona en diferentes sistemas operativos y navegadores.
- Sensibilidad a mayúsculas y minúsculas: Las variables nombre y Nombre son diferentes.
- Lenguaje de alto nivel: Tiene una sintaxis sencilla y fácil de comprender.

2. Estructuras de control (condicionales y bucles)

Las estructuras de control permiten tomar decisiones y repetir instrucciones.

Condicionales

if

Ejecuta un bloque de código si una condición es verdadera.

```
let edad = 20;

if (edad >= 18) {
  console.log("Mayor de edad");
}
```

if...else

Permite ejecutar un bloque diferente cuando la condición es falsa.

```
if (edad >= 18) {  
    console.log("Mayor de edad");  
} else {  
    console.log("Menor de edad");  
}
```

switch

Se utiliza cuando existen múltiples opciones posibles.

```
let dia = 2;  
  
switch(dia){  
    case 1:  
        console.log("Lunes");  
        break;  
    case 2:  
        console.log("Martes");  
        break;  
    default:  
        console.log("Otro día");  
}
```

Bucles

for

```
for(let i = 1; i <= 5; i++){  
    console.log(i);  
}
```

while

```
let i = 1;  
  
while(i <= 5){  
    console.log(i);  
    i++;  
}
```

do...while

```
let numero = 1;

do{
  console.log(numero);
  numero++;
}while(numero <= 5);
```

Los bucles permiten repetir instrucciones mientras una condición se cumpla.

3. Tipos de datos (primitivos y referencias)

Datos primitivos

Son valores simples que se almacenan directamente.

- Number (números)
- String (cadenas de texto)
- Boolean (verdadero o falso)
- Undefined (variable sin valor)
- Null (ausencia intencional de valor)
- BigInt (enteros muy grandes)
- Symbol (identificadores únicos)

Ejemplo:

```
let nombre = "Juan";
let edad = 22;
let activo = true;
```

Datos por referencia

Son objetos almacenados mediante referencias en memoria.

- Object
- Array
- Function
- Date
- Map
- Set

Ejemplo:

```
let alumno = {
  nombre: "Ana",
  edad: 20
};
```

La diferencia principal es que los datos primitivos almacenan el valor directamente, mientras que los datos por referencia almacenan la dirección donde se encuentra el objeto.

4. Funciones (declaradas, expresadas y flechas)

Las funciones son bloques de código reutilizables que realizan una tarea específica.

Función declarada

```
function saludar(){
  console.log("Hola");
}
```

Función expresada

```
const saludar = function(){
  console.log("Hola");
};
```

Función flecha (Arrow Function)

```
const saludar = () => {
  console.log("Hola");
};
```

Las funciones flecha ofrecen una sintaxis más corta y son muy utilizadas en JavaScript moderno.

5. Objetos y Arrays

Objetos

Un objeto almacena información mediante pares **clave-valor**.

```
const persona = {
  nombre: "Carlos",
  edad: 25,
  ciudad: "Tepic"
};
```

```
console.log(persona.nombre);
```

Arrays

Un array almacena varios elementos en una sola variable.

```
const colores = ["Rojo", "Azul", "Verde"];
```

```
console.log(colores[0]);
```

Los arrays pueden contener números, texto, objetos e incluso otros arrays.

6. Eventos y manipulación del DOM

Eventos

Los eventos son acciones que ocurren en la página web, por ejemplo:

- Clic del mouse.
- Presionar una tecla.
- Mover el cursor.
- Enviar un formulario.
- Cargar una página.

Ejemplo:

```
document.getElementById("boton").addEventListener("click", function(){  
    alert("Botón presionado");  
});
```

Manipulación del DOM

El **DOM (Document Object Model)** representa la estructura de una página HTML como un árbol de elementos. JavaScript puede modificar esos elementos.

Ejemplo:

```
document.getElementById("titulo").textContent = "Bienvenido";
```

Con el DOM es posible:

- Cambiar texto.
- Cambiar imágenes.
- Modificar estilos.
- Crear nuevos elementos.
- Eliminar elementos.
- Responder a eventos del usuario.

7. Asincronía (Callbacks, Promesas y async/await)

La asincronía permite ejecutar tareas que tardan tiempo sin detener el resto del programa.

Callbacks

Una función que se ejecuta cuando otra termina.

```
setTimeout(function(){  
    console.log("Hola");  
}, 2000);
```

Promesas (Promises)

Representan el resultado futuro de una operación que puede completarse correctamente o fallar.

```
const promesa = new Promise((resolve, reject)=>{  
    resolve("Operación exitosa");  
});  
promesa.then(resultado=>{  
    console.log(resultado);  
});
```

Async/Await

Es una forma moderna y más sencilla de trabajar con promesas.

```
async function obtenerDatos(){  
    let respuesta = await fetch("https://api.ejemplo.com");  
    let datos = await respuesta.json();  
    console.log(datos);  
}
```

Las ventajas de async/await son:

- Código más limpio.
- Más fácil de leer.
- Facilita el manejo de errores.
- Evita el exceso de callbacks.

Cuadro Sinoptico

{.js}

JavaScript

JavaScript

JS

CARACTERÍSTICAS BÁSICAS

- **Interpretado:** se ejecuta directamente sin compilar
- **Dinámico:** tipos de datos flexibles
- **Multiparadigma:** Soporta POO y funcional
- **Débilmente tipado:** Permite operaciones entre distintos tipos sin error

EJEMPLO

Dinámico
`let x = {x: 'A'};`
Débilmente tipado
`'5' + 2 → '52'`

ESTRUCTURA DE CONTROL

- **Condicionales:** if, else, switch
- **Bucles:** for, while, forEach
- **Iteradores:** for...of, for...in, .map(), filter()
- **Control de flujo:** break, continue, return

EJEMPLO

```
if (edad >= 18) {  
  console.log("Mayor de edad");  
}  
for (let i=0; i<3; i++) {  
  console.log(i);  
}  
[1,2,3].forEach(n =>  
  console.log(n));  
for (let i=0; i<5; i++) { if  
(i===3) break; }
```

TIPO DE DATOS

- **Primitivos:** number, string, boolean
- **Referencias:** object, array, function

EJEMPLO

```
let nombre = "Erick"; // string  
let persona = { nombre: "Erick",  
  edad: 25 };
```

FUNCIONES

- **Declarada:** Se define con function y nombre propio.
- **Expresada:** Se guarda en una variable/constante.
- **Flechas:** Sintaxis corta con =>, hereda this.

EJEMPLO

```
function suma(a,b){ return  
  a+b; }  
const resta = function(a,b){  
  return a-b; };  
  
const multiplica = (a,b) =>  
  a*b;
```

OBJETOS

- **Objetos:** Estructuras con pares clave: valor.
- **Arrays:** Listas ordenadas por índice.

EJEMPLO

```
let coche = { marca: "Toyota",  
  modelo: "Corolla" };  
  
let numeros = [10,20,30];
```

EVENTOS Y MANIPULACIÓN DEL DOM

- **addEventListener:** Permite escuchar acciones del usuario
- **querySelector:** Selecciona elementos del DOM por id, clase o etiqueta
- **Manipulación del contenido:** Cambiar texto, estilos o estructura de la página.

EJEMPLO

```
document.querySelector("#btn")  
  .addEventListener("click",  
    () => alert("Hola"));  
  
let titulo =  
  document.querySelector("#tit  
ulo");  
  
titulo.textContent = "Nuevo  
titulo";
```

ASINCRONÍA

- **callbacks:** Función dentro de otra para ejecutar después.
- **promesa:** Manejan resultados futuros.
- **async/await:** Sintaxis más legible para promesas.

EJEMPLO

```
setTimeout(() =>  
  console.log("Hola"),  
  1000);  
  
fetch("api/datos").then  
  (res => res.json());  
  
async function  
  cargar() { let datos =  
    await  
    fetch("api/datos"); }
```

Erick Alejandro Miramontes Huerta